How your website works

Yūki's Sacred Space

A guide for you — and for whoever
looks after it next

yukis.space

Last updated August 2026

Contents

Part one

Using your website — no technical knowledge needed

1. What you have

Your site lives at **yukis.space**. It also answers on `www.yukis.space` and on the older `yukispace.duckdns.org`, so any link already shared with anyone still works. All of them are secure (the padlock in the browser).

The site has six public pages: the home page, your about page, your writing, the free guides, a brand page, and the five printable workbooks. Booking and payment are still handled entirely by Square — every "book" button opens your existing Square page. **Nothing about your Square setup changed**, and money never passes through this website.

> **The one thing to remember:** the website advertises your prices, Square charges them. If you change one, change the other, or someone books at a price you are not charging.

2. Signing in

Go to **yukis.space/admin** and sign in with your email and password. If you have not signed in, it will send you to the login screen automatically.

The admin only exists over a secure connection. If you ever type `http://` instead of `https://` it will look like the page does not exist — that is deliberate, not a fault. Use the link above and it will always be right.

> **Changing your password** is the one thing the admin cannot do yet. Ask whoever looks after the site and it takes about a minute. Worth doing, since the password you were given was generated for you rather than chosen by you.

3. The admin, tab by tab

3.1 Resources

This is the list of free guides shown on your *resources* page.

Each entry has a title, a category (like "cbt workbook"), a short description, and a link. **Nothing appears on the site until you tick "published"** — so you can write something, leave it as a draft, and come back to it.

Sort order controls the order they appear in. Lower numbers come first, and they go up in tens so you can slot something in between later without renumbering everything.

You can also upload a file (a PDF or an image) instead of linking to a page. Use that if you write a worksheet in Word and want to hand it out as a PDF.

The five workbooks that already exist are ordinary web pages, not database entries you can rewrite here. Editing the words inside a workbook needs a developer; changing how it is *listed* does not.

3.2 Testimonials

Quotes from people you have worked with, shown on your home page.

Same rule: **nothing shows until you tick "published"**, which is deliberate. Quoting someone by accident is the one mistake here that cannot be undone by editing a row afterwards.

> **Two things to be careful about.**
>
> *Permission.* A public Facebook review is already public. A private message is not. Ask before quoting a DM, and ask what name they want on it.
>
> *Health claims.* Be wary of publishing a quote that says you cured, reversed, or treated a medical condition — even though the person wrote it themselves, the risk lands on your business, not theirs. Quotes about how someone *felt* (slept better, calmer, clearer) are fine. Quotes claiming a specific medical outcome are the ones to leave unpublished. There is a disclaimer under the testimonials on your site, and it helps, but it does not cover everything.

3.3 Mailing list

Everyone who has signed up through your site.

People land here as **pending** until they click the confirmation link in their email. Only **confirmed** people ever receive a newsletter — that is what keeps you out of spam folders.

You can search by email, name or notes, and filter by status, by where they signed up, and by date. **Everything you do next respects that filter:**

- **Copy addresses** puts the matching emails on your clipboard, so you can paste them into any other mail program.
- **Download CSV** gives you a spreadsheet of the same list.
- **Add someone** is for people who gave you their email in person. They go straight in as confirmed, because you already have consent.

People who unsubscribe stay in the list marked "unsubscribed" rather than being deleted. That is on purpose — it is what stops them being added again by accident later.

3.4 Mail blast

Where you write and send a newsletter.

The top section, *ready to write about*, fills itself in. Every time you publish a testimonial or a guide, or post something new on Medium, it appears here as a suggestion. Click **write newsletter** and it drafts one for you — subject, body and button already filled in. **Then edit it into your own words**, because the draft is a starting point, not your voice.

Before sending, always:

1. **Preview** — opens exactly what people will receive.
2. **Test send** — sends one copy to any address you choose. Send it to yourself and read it on your phone.
3. **Send to everyone** — goes to everyone confirmed.

> **Sending cannot be undone.** There is no recall. Test send first, every time. Once it has gone out, that newsletter locks and cannot be edited — if you need to change it, make a new one.

3.5 Visits

How many people came, what they looked at, and what they did.

The four numbers at the top are visitors, page views, book clicks and signups, each compared against the previous period.

Book clicks is the number to watch. Two hundred visits with no book clicks means very different things from forty visits with six. Visits tell you whether people are arriving; book clicks tell you whether the site is convincing them.

Where they came from becomes useful the day you post the link to Facebook — you will see the traffic arrive and know it worked.

Nothing here tracks individual people. There are no cookies, nothing is sent to Google, and no one can be identified or followed from one day to the next. Anyone browsing with "do not track" turned on is not counted at all. That is also why your site does not need one of those irritating cookie banners.

4. What you can change yourself

You can do this in the admin	This needs a developer
Add, edit, reorder and publish resources	The words inside a workbook
Add, publish and unpublish testimonials	Your prices
Upload PDFs and images	Your about page and its wording
Add people to the mailing list	The session descriptions
Write, preview, test and send newsletters	Page layout, colours, fonts
Watch your visitor numbers	Adding a new page

Your prices, your about page and your session descriptions live in the website files rather than the admin. That is a deliberate trade: those change rarely, and keeping them out of the database means there is far less that can break.

5. If something looks wrong

- **The admin will not load.** Check you are on `https://` and that the address is exactly `yukis.space/admin`.
- **You changed something and the site looks the same.** Refresh the page properly (Ctrl+F5, or Cmd+Shift+R on a Mac).
- **The resources page shows the old list.** The site keeps working from a saved copy if the database is briefly unavailable, so this usually fixes itself. If it persists, it needs a developer.
- **Someone says an email went to spam.** Expected occasionally while sending from a Gmail address. Part two explains what fixes it properly.
- **Anything else.** Note exactly what you clicked and what happened, and hand part two of this document to whoever helps you.

Part two

Technical handover — for a developer

This section assumes the reader has never seen the project. It is written so someone can take it over without needing to ask questions.

6. Architecture at a glance

A mostly-static marketing site on a single EC2 instance, with a small Node application behind nginx providing an admin portal, a database-backed resources page, a mailing list and first-party analytics.

Layer	What it is
Static site	Hand-written HTML/CSS/JS, no framework, no build step for the markup
Page generation	Three Python scripts write the sub-pages and workbooks
Web server	nginx — serves static files, proxies a handful of routes
Application	Node 22 + Express under PM2, bound to loopback only
Database	MySQL 8.4
Payments	Square, entirely external. No card data ever touches this server

7. Access a new developer needs

Thing	Detail
AWS account	Owns the EC2 instance, Elastic IP and the Route 53 hosted zone
SSH key	`Yukis.pem`. On Windows, permissions are set with `icacls`, not `chmod` — see below
GitHub	`https://github.com/penpro/Yuki` (public)
Domain	`yukis.space`, registered in Route 53
Gmail account	`yukissacredspace@gmail.com` — sends the site's email
Square	Hers. Nothing here integrates with it beyond outbound links

8. AWS

Instance	`i-0f7d9894c082e9a49`, `t3.micro`
Region / AZ	`us-east-2b`
OS	Ubuntu 26.04 LTS
Resources	2 vCPU, 908 MB RAM, 6.7 GB disk
Elastic IP	`52.14.87.6` — static, must stay associated
Security group	Inbound 22 (SSH), 80, 443 only

The Elastic IP is load-bearing. A plain EC2 public IP changes every time the instance stops and starts, and released addresses are reassigned to other customers within hours. If it is ever detached, DNS ends up pointing at somebody else's server. Leave it attached; AWS bills for unattached ones anyway.

SSH:

```
ssh -i ~/.ssh/Yukis.pem ubuntu@52.14.87.6
```

On Windows, `chmod 600` on the key does nothing useful — OpenSSH reads NTFS ACLs, not POSIX bits, and will still reject the key. Use:

```
icacls key.pem /inheritance:r
icacls key.pem /grant:r "%USERNAME%:(R)"
```

9. Domain and DNS

`yukis.space` is registered in Route 53, in the same AWS account. Its hosted zone holds two A records — the root and `www` — both pointing at `52.14.87.6`.

`yukispace.duckdns.org` is a free dynamic-DNS name kept alive so earlier links do not break. It is on the same certificate and can be retired whenever.

If the domain is transferred to another AWS account, the hosted zone does *not* move with it. Recreate the zone and both A records on the receiving account, and confirm resolution before letting the old zone go.

10. The server

Repo checkout	`/home/ubuntu/yuki`
Web root	`/var/www/yuki`
Uploads	`/var/www/yuki-uploads` (outside the web root)
nginx site	`/etc/nginx/sites-available/yuki`
nginx snippets	`/etc/nginx/snippets/yuki-*.conf`
App process	PM2, `yuki-backend`, `127.0.0.1:3001`
App config	`/home/ubuntu/yuki/backend/.env` (0600, never committed)
Certificates	Let's Encrypt, cert name `yukis.space`, auto-renewing
Hardening	ufw, fail2ban (sshd jail), unattended security upgrades
Swap	1.5 GB file, `vm.swappiness=10`

Why there is swap and a tuned MySQL. The instance has under 1 GB of RAM and shipped with no swap. MySQL 8.4's defaults assume far more — `performance_schema` alone reserves over 100 MB. Installed untuned, `mysqld` gets OOM-killed days later under load, which presents as a mystery crash. `/etc/mysql/mysql.conf.d/zz-low-memory.cnf` caps the buffer pool at 96 MB and disables `performance_schema`. Raise these only if the instance is resized.

11. Repository layout

`index.html`	Home page. **Hand-written, not generated**
`build-pages.py`	Generates about / writing / resources / brand
`build-guides.py`	Generates the five workbooks
`build-brand.py`	Writes the logo SVGs
`assets/`	CSS and JS. Design tokens are at the top of `styles.css`
`backend/`	Express app. **Never deployed to the web root**
`deploy/`	All ops scripts and nginx config
`guides/`, `brand/`	Generated output, committed

Editing a generated page by hand works, but re-running its script overwrites it. Change the script instead.

12. Deploying

Everything is push-then-pull. Nothing is built or uploaded from a workstation.

1. Commit and push to GitHub.
2. On the server: `cd ~/yuki && ./deploy/pull-and-deploy.sh`

`pull-and-deploy.sh`	Everyday: git pull, then publish static files
`deploy.sh`	Publish only
`deploy-backend.sh`	Migrations, npm install, PM2 restart, health check
`server-setup.sh`	nginx, TLS, snippets. Idempotent
`provision-stack.sh`	One-time: swap, MySQL, Node, PM2, hardening
`db/migrate.sh`	Applies `db/migrations/*.sql`, tracked in `schema_migrations`

Changing the domain or the nginx config:

```
EXTRA_DOMAINS="www.yukis.space yukispace.duckdns.org" \
CERTBOT_EMAIL="you@example.com" ./deploy/server-setup.sh yukis.space
```

13. Database

Schema `yuki`. Migrations are numbered and tracked; never edit one that has already run — write a new one.

`admin_users`	bcrypt hashes only. Created by `backend/create-admin.js`
`sessions`	Managed by express-mysql-session
`resources`	The catalogue behind `/resources`
`testimonials`	Home page quotes
`subscribers`	Mailing list, with confirm and unsubscribe tokens
`campaigns`, `campaign_sends`	Newsletters and per-recipient delivery
`page_views`, `events`	Analytics

Two database users: MySQL `root` via unix socket (used by migrations, no password anywhere), and `yuki_app` over TCP with rights scoped to this schema and no `DROP` or `GRANT`. The application uses the latter.

14. Email

Currently sends through Gmail SMTP as `yukissacredspace@gmail.com`, authenticated with a Google *app password* (which requires 2-Step Verification on that account).

> **This is a deliberate stopgap, not the end state.** It suits a mailing list of a few people. It caps at roughly 500 messages a day, mail arrives from a `gmail.com` address rather than the domain, and any spam complaints land against a personal Google account.
>
> Before any real list-building, move to a transactional relay (Brevo, Mailgun, Postmark or SES), send from `hello@yukis.space`, and add **SPF, DKIM and DMARC** records to the Route 53 zone. Without those three records, Gmail and Outlook treat the sender as unproven regardless of content. Only `SMTP_*` and `MAIL_*` values in `.env` need to change.

Confirm and unsubscribe links are built from `APP_BASE_URL`. If the domain ever changes, that must change with it, or every link in every email points at the old host.

15. Deliberate decisions — please do not undo these

- **The admin only exists when a domain is configured.** nginx pulls the admin routes in through a wildcard include that matches no file until `server-setup.sh` runs with a real domain. That is the same precondition as a certificate, so a login form can never be served over plain HTTP.
- **Node binds `127.0.0.1` only.** The port stays unreachable even if a security group rule is opened by mistake.
- **Login gives one generic error** and burns an equivalent bcrypt compare when the account does not exist, so neither the message nor the response time reveals which addresses are real.
- **Uploads use an extension allowlist and server-generated random filenames.** The client-supplied name never reaches disk.
- **Unsubscribe is the least protected route on the site** — no auth, one click, GET or POST. Making opting out harder than opting in is what produces spam complaints.
- **Analytics stores no IP or user-agent.** Visitors are counted by a daily-rotating one-way hash. This is why no cookie banner is required.
- **Content never depends on JavaScript.** A `<noscript>` block neutralises the scroll-reveal CSS; without it a failed script load renders a blank page.

16. Traps that have already caught someone

Each of these cost real debugging time. They are listed so they do not have to be rediscovered.

1. **`server-setup.sh` overwrites certbot's config.** Copying the repo's nginx file over the live one discards the 443 block. The script now reinstalls TLS automatically — keep that behaviour.
2. **Never pipe certbot through `grep`.** The pipeline reports grep's exit status, so a failed certificate install looks like a success. This once left a broken config on disk while nginx served the last good one from memory — working until the next reboot.
3. **Duplicate nginx directives fail `nginx -t` outright.** Setting `proxy_connect_timeout` in a location that also includes the proxy snippet breaks the whole config.
4. **`package-lock.json` must stay committed.** `npm ci` installs from it and hard-fails when it drifts, which aborts the deploy before the PM2 restart and leaves old code running.

5. **`backend/` must stay excluded from the web root.** It was briefly rsynced there, making server source downloadable.
6. **`/assets/` is served `no-cache` on purpose.** The filenames carry no content hash, so a long cache meant fixes took a day to reach returning visitors. Add hashed filenames before caching aggressively.
7. **Square booking links cannot be deep-linked.** Their buttons are JavaScript-driven with no plain URLs, so every book button points at the appointments list. Per-service URLs must come from her Square dashboard.

17. Running costs

EC2 `t3.micro` (outside free tier)	~$7.50 / month
Public IPv4 address	~$3.60 / month
Route 53 hosted zone	$0.50 / month
`yukis.space` registration	annual, varies
TLS certificate	free
Email (current setup)	free
Approximate total	**~$12 / month**

18. Handover checklist

If the project passes to someone else, rotate everything — previous access should stop working.

1. Create a new EC2 key pair and replace `authorized_keys`; retire `Yukis.pem`.
2. Rotate `SESSION_SECRET` in `.env` (this signs out everyone).
3. Rotate the `yuki_app` MySQL password in both MySQL and `.env`.
4. Revoke the Google app password and issue a new one.
5. Reset the admin password with `backend/create-admin.js`.
6. Transfer or share the AWS account, the Route 53 domain and the GitHub repository.
7. Confirm the Elastic IP is still associated, and that certificate auto-renewal is running (`systemctl list-timers | grep certbot`).

The site is deliberately boring to run: static files, one small application, one database, and scripts that are safe to re-run. Almost everything can be fixed by pushing to GitHub and running `pull-and-deploy.sh`.